as 88%-99% paraphrases achieve a P-SP score higher than the median score of human-written paraphrases. High semantic preservation is supported by careful human evaluations in Appendix C.2. Overall, we find that watermarking is the most resilient detector to paraphrasing.

Non-watermarking detectors are generally ineffective. We observe that all detectors apart from watermarking struggle with text generated by larger models like OPT-13B and GPT-3.5, achieving detection accuracies $< 50\%$. While DetectGPT is effective on the smaller GPT2-XL model (74.9% on long-form QA), its accuracy drops to just 29.8% on OPT-13B. Furthermore, GPTZero and RankGen perform the worst among the five detectors on all tested LMs (Table 1), as they are only able to detect < 15% of non-paraphrased AI-generated text. Thus, we recommend against using these detectors.

ROC plots confirm the trends at different false positive rates. In Figure 4, we plot the detection accuracy (true positive rate) at different values of FPR between 0% and 1% for GPT2-XL. Overall, paraphrasing significantly drops detection rates across all FPR thresholds (more plots in Appendix H).

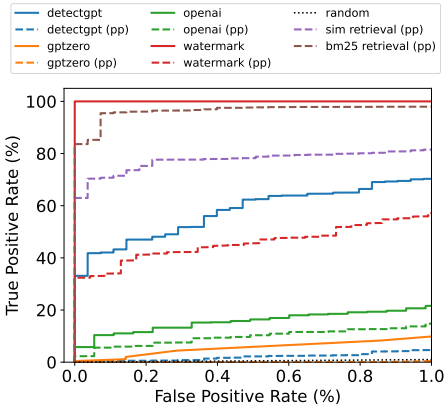

Figure 4: ROC plots (0-1% FPR) for GPT2-XL using different detectors, before (solid lines) and after paraphrasing (dashed). Paraphrasing reduces detection rate across FPRs, and our detector *retrieval* detects paraphrases best; full plots in Appendix H.

4.4 Alternative paraphrasing attacks

Paraphrasing multiple times: Our presented attacks use just a single paraphrase generated by DIPPER to evade detection. A simple way to further improve the effectiveness of a paraphrase attack is to sample multiple times[12] from DIPPER and choose a paraphrase that evades the detector while also preserving semantics. We do not perform this attack as it can only be done if an attacker has access to a detector, which may be a strong assumption (see Appendix A.2). That being said, using multiple paraphrase samples can make the attacks even more potent against publicly available detectors.

Non-DIPPER paraphrasers: A second alternative is to use non-DIPPER paraphrasers that operate at the sentence level. These models can be deployed for long-form text inputs by paraphrasing the inputs sentence by sentence, ignoring prompt context. While the concurrent work of Sadasivan et al. [2023] shows that this method can also evade detection, our ablations in Appendix C show that non-contextual versions of DIPPER have lower quality and are less compatible with the prompt as DIPPER paraphrasers. Moreover, most existing paraphrasers lack fine-grained diversity control and multi-sentence input support (survey in Appendix D.1), two desired properties from an attacker's point of view: attackers want to modify long multi-sentence responses *just enough* to evade detection.

A more interesting option is to use an LLM like ChatGPT to perform few-shot contextual paraphrasing. While this method is likely to provide accurate paraphrases,[13] they may be detectable by strategies like watermarking (whether using the same API as the original LLM or a different one). We thus expect a sophisticated adversary to use their own private paraphraser (like DIPPER) to evade detection.

5 Defense against paraphrase attacks using retrieval

In Section 4.3, we showed that paraphrasing is an effective attack against AI-generated text detectors. How can LLM API providers defend against these attacks? In this section, we propose *retrieval* over previously-generated sequences as a defense against paraphrase attacks. At a high level (Figure 5), an API provider first stores every sequence generated by their LLM in a database. The API provider offers an interface that allows users to enter candidate AI-generated text as a query. The interface searches

[12]Precisely, compute $f_{\text{dipper}}(x)$ for different random seeds while sampling text. Alternatively, an attacker could also compute $f_{\text{dipper}}(f_{\text{dipper}}(...f_{\text{dipper}}(x)))$, but this will lead to excessive semantic drift from x.

[13]In initial experiments, we observed that DIPPER performs competitively with the much larger and more powerful GPT-3.5 davinci-003 model in terms of paraphrase quality, and significantly better at controlling diversity. This finding shows that specialized smaller models can outperform LLMs in paraphrasing tasks.

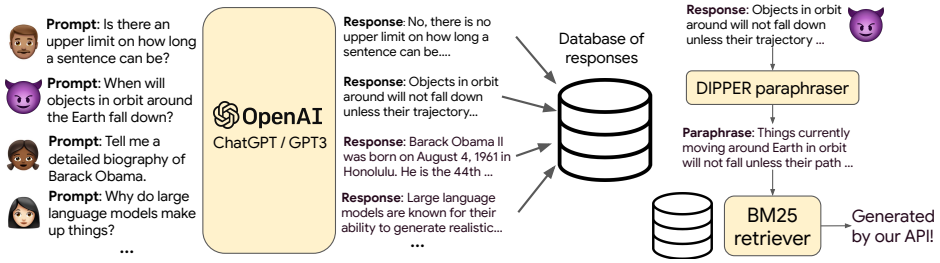

Figure 5: An illustration of AI-generated text detection with retrieval. Several users (including the attacker, shown as the purple emoji) feed prompts to the API which are collectively added to a private API-side database. Candidate queries are compared against this database using a retriever like BM25.

over the entire database of previously-generated text, trying to find a sequence that approximately matches the content of the input query. This search can be done using a semantic similarity scorer like P-SP [Wieting et al., 2022] or a retriever like BM25 [Robertson et al., 1995]. Since paraphrasing approximately preserves input semantics, we expect such a defense to still be able to map paraphrased generations to their source. We formalize our detector in Section 5.1, and then conduct a controlled comparison with competing detectors in Section 5.2. We evaluate retrieval-based detection at scale using a large retrieval corpus of 15M generations in Section 5.3. In Appendix A we extensively discuss limitations of retrieval-based detection and share ideas for enabling further scaling.

5.1 Formulating the retrieval defense

Let f_{LM} be an LLM API (e.g., GPT-3.5) that takes a prompt x as input and returns a continuation y. Let f_{ret} be an encoder (e.g., TF-IDF, neural network) that embeds variable-length sequences into fixed-size vectors that represent the input semantics. Then, we do the following:

Building the database: Let $x_1, ..., x_N$ be the set of prompts that have been fed as input to the API in the past with $y_i = f_{\text{LM}}(x_i)$ being the LLM output. Here N can potentially be very large for popular APIs (we study up to $N = 15\text{M}$). We construct our database $\mathbf{Y} = [\mathbf{y}_1, ...\mathbf{y}_N]$ by encoding every LLM API output with our retrieval encoder, or $\mathbf{y}_i = f_{\text{ret}}(y_i)$. The database $\mathbf{Y}$ is dynamically updated and stored on the API side. It is inaccessible to clients except via the API described in the next step.

Querying the database: Let y' be a candidate text and $\mathbf{y}' = f_{\text{ret}}(y')$ its encoded vector. Suppose a client wishes to know whether y' was generated by the API f_{LM}. The API provider can check this by seeing whether the maximum similarity score of y' to an entry in the database exceeds some detection threshold T chosen by the API provider:

$$\text{output} = \text{score} > T, \text{ where score} = \max_{i \in \{1,..N\}} \frac{\mathbf{y}' \cdot \mathbf{y}_i}{|\mathbf{y}'| \, |\mathbf{y}_i|}$$

We expect unperturbed machine-generated text to always get a score of 1.0, while paraphrasing the text may lower the detection score. Hence, lowering T will increase the detection rate of heavily-paraphrased text but also increase the false positive rate (i.e., human-written text that resembles sequences previously generated by the LLM API can be falsely flagged). Since N can be very large, the score can also be approximated using efficient nearest neighbor libraries like FAISS [Johnson et al., 2019]. However, in this work we only compute exact inner products.

As the **retriever** f_{ret}, we experiment with two choices: P-SP [Wieting et al., 2022] and BM25 [Robertson et al., 1995]. We implement BM25 using the `retriv` library from Bassani [2022]. In order to normalize and calibrate BM25 scores, we compute the F1-score unigram token overlap [Rajpurkar et al., 2016] between the candidate y' and the best retrieval $y*$ to get a detection score in $[0, 1]$.

5.2 Controlled comparisons of retrieval with other AI-generated text detectors

First, we conduct a controlled comparison between the detection algorithms evaluated in Section 4.3 and our retrieval method on long-form question answering.[14] We construct three kinds of databases,

1. 3K sequences generated by a specific LM for one of the tasks;

[14] Corresponding results in open-ended text generation on Wikipedia prompts are provided in Appendix G.1.

Table 2: A comparison of retrieval against other detectors on long-form QA (300 generated tokens). Our detector outperforms baselines (at 1% FPR) even with the most diverse paraphrases (+60L,O).

	GPT2-XL			OPT-13B			GPT-3.5 (davinci-003)		
	Original	+ 60L	+ 60 L,O	Original	+ 60L	+ 60 L,O	Original	+ 60L	+ 60 L,O
Watermark [2023a]	100.0	71.1	55.8	100.0	65.5	51.4	-	-	-
DetectGPT [2023]	74.9	15.8	7.6	29.8	3.2	1.5	1.0	0.0	0.0
OpenAI [2023a]	59.2	31.3	32.7	33.5	21.6	21.6	40.5	40.1	38.1
(Ours) Retrieval over corpus of 3K generations from model itself, with retriever:									
SP	100.0	95.6	87.7	100.0	94.8	85.3	100.0	94.2	85.1
BM25	100.0	99.2	97.8	100.0	99.3	97.3	100.0	98.6	96.2
(Ours) Retrieval over corpus of 9K generations pooled from all three models, with retriever:									
SP	100.0	88.9	75.4	100.0	89.6	76.4	100.0	93.8	84.6
BM25	100.0	98.3	95.2	100.0	98.5	94.4	100.0	98.5	96.0
(Ours) Retrieval over 43K ShareGPT responses + corpus of 3K generations from model itself, with retriever:									
SP	100.0	94.0	84.8	100.0	94.2	84.7	100.0	94.1	84.9
BM25	100.0	98.9	97.5	100.0	99.0	97.3	100.0	98.4	95.5

2. 9K sequences formed by concatenating the generations from all three LMs in this paper;
3. 46K sequences constructed by combining the 3K sequences from (1) with 43K LLM responses from ShareGPT-Vicuna.[15]

We expect (2) to be a more difficult test for our method than (1), since the retriever needs to distinguish between multiple generations from different models given the same prompt. On the other hand, (3) denotes a more real-world setting with several diverse LLM-generated outputs from ShareGPT. Next, we perform retrieval over this corpus using different types of queries: the original AI-generated text, its DIPPER paraphrase, and human-written text (each query with at least 50 tokens).

Table 2 shows that **across all LMs, retrieval is a much more effective detector than baseline detectors**. On unperturbed AI-generated text, retrieval has a 100% detection accuracy due to exact match with the retrieval corpus. On paraphrased text, retrieval with BM25 is quite effective, detecting 97.8% of the highest-diversity paraphrases (L60, O60) on GPT2-XL, 97.3% on OPT-13B and 96.2% on GPT-3.5 in long-form question answering. This is significantly better than the next best alternative with competing detectors (55.8%, 51.4%, 38.1%). Even on our harder augmented databases, detection rates continue to be high: 95.2%, 94.4%, 96.0% for the 9K augmented database; 97.5%, 97.3%, 95.5% for the ShareGPT augmented database. Finally, we observe that BM25 is a more effective retriever than P-SP, scoring 95.2% vs 75.4% on the augmented setting in GPT2-XL. These trends are consistent across different FPR thresholds, as shown in Figure 4.

In Appendix G.2, we additionally observe promising preliminary results that show the effectiveness of retrieval against **text mixing attacks** [Kirchenbauer et al., 2023b].

5.3 Is retrieval an effective detector with a large retrieval corpus?

In the previous section, we conducted experiments using the set of 9K sequences generated by all three models as the retrieval corpus. However, this is more of a toy experiment: in practice, a popular LLM API may serve millions of queries a day. As the corpus grows larger, the false positive rate (i.e., human-written text falsely detected as AI-generated) will grow. How well do retrieval-based detectors scale? To answer this question, we need access to a large corpus of AI-generated text. We utilize the training data used to train RankGen [Krishna et al., 2022a], which contains over 70M AI-generated sequences. We use the Project Gutenberg and Wikipedia splits of the training data, each of which contain 15M sequences generated by a T5-XXL model [Raffel et al., 2020] fine-tuned on the different documents in the same domain. We discard generations which are shorter than 50 tokens, and paraphrase a subset of 2K generations to evaluate retrieval.

Retrieval is effective even with a corpus size of 15M generations. In Figure 6a, we plot the detection accuracy as a function of retrieval database size. Overall, we observe that detection

[15] huggingface.co/datasets/anon8231489123/ShareGPT_Vicuna_unfiltered